CivicLoop

An open-weight AI agent for persistent civic problem management.

CivicLoop is a civic-tech project designed to help residents keep related civic complaints, supporting evidence, guidance, and follow-up actions together in one persistent case history. The initial scope focuses on Bengaluru, Karnataka, with early use cases such as garbage accumulation and blocked drainage.

> **Project status:** This repository contains an evolving prototype and implementation scaffolding. Features and integrations should be treated as available only where they are implemented and verified in the code. CivicLoop is intended to complement existing civic reporting channels, not replace official government portals.

Contents

- [Problem](#problem)
- [Project goals](#project-goals)
- [Planned workflow](#planned-workflow)
- [Technology stack](#technology-stack)
- [Repository structure](#repository-structure)
- [Getting started](#getting-started)
- [Configuration](#configuration)
- [Development and testing](#development-and-testing)
- [Security and privacy](#security-and-privacy)
- [Contributing](#contributing)
- [Acknowledgements](#acknowledgements)
- [License](#license)

Problem

A resident may report an issue through one channel and then need to track receipts, complaint references, screenshots, status updates, and follow-up conversations elsewhere. An administrative status marked “closed” also does not necessarily prove that the physical problem has been resolved.

CivicLoop explores how to keep this information connected, help residents understand evidence-based next steps, and record whether an outcome has been verified.

Project goals

The project is designed around the following goals:

- **Persistent case history:** keep complaint records, evidence references, and timeline events connected.
- **Evidence-aware follow-up:** distinguish a status reported by a source from a resident's observation of the real-world outcome.
- **Grounded guidance:** use curated civic information and show source references when available.

- **Bounded AI assistance:** use an AI model to help interpret requests and suggest next steps while backend code enforces validation, permissions, and state changes.
- **Durable workflows:** support follow-up tasks and controlled retries rather than relying only on an active browser session.
- **Privacy by default:** avoid exposing personal information or private evidence unnecessarily.
- **Human approval:** require explicit approval before consequential external actions.

These are project goals; consult the current code and tests to determine which capabilities are implemented.

Planned workflow

- A resident enters complaint details or provides a complaint receipt, screenshot, reference, or observation.
- The system can extract candidate information where supported and ask the resident to review uncertain fields.
- Related records and relevant official guidance may be suggested.
- The resident confirms or rejects proposed links between records.
- CivicLoop proposes a next step with source context where available.
- Any consequential action requires the appropriate permission and human approval.
- A follow-up task can be saved for later handling.
- If a source reports closure, the resident's verification is recorded separately from the source's administrative status.

Automatic submission to government portals or live status checks should not be assumed unless an authorised, working integration is present.

Technology stack

The repository configuration indicates the following technologies and components:

Area	Technology / component	Purpose
Frontend	React, TypeScript, Vite	Web interface
Backend API	Python, FastAPI, Pydantic	API endpoints and request validation
Database	PostgreSQL with `pgvector`	Persistent application data and vector support
Database migrations	Alembic (configured/planned in the project)	Schema changes
Background jobs	Worker service; Redis/Celery are part of the proposed architecture where configured	Follow-up processing and retries
AI inference	OpenAI-compatible model endpoint for open-weight models	Model access through configurable endpoints
Vision / extraction	Configurable vision model and document-processing components	Candidate field extraction from supported evidence
Local environment	Docker Compose	Coordinated local services
Testing	`pytest` and frontend tests where configured	Validation of application behaviour

The exact model variants, integrations, and individual features may require additional setup. Check the source code and configuration files before assuming a component is fully operational.

Repository structure

The repository currently includes these top-level areas:

```
Civic-Loop/
■■■ backend/          # Backend API and application logic
■■■ frontend/         # Web frontend
■■■ worker/           # Background worker
■■■ knowledge/        # Knowledge/source-related assets
■■■ eval/             # Evaluation-related files
■■■ tests/            # Tests
■■■ uploads/evidence/ # Evidence-related directory; do not commit real personal data
■■■ .env.example      # Example environment configuration
■■■ docker-compose.yml
```

Individual files and subdirectories may evolve as development continues.

Getting started

Prerequisites

- Git
- Docker and Docker Compose
- An internet connection for downloading container images and dependencies
- A compatible OpenAI-style model endpoint, either local or hosted, configured for the model names used by the project

1. Clone the repository

```
git clone https://github.com/Rishank-07/Civic-Loop.git
cd Civic-Loop
```

2. Create your local environment file

```
cp .env.example .env
```

Review `.env` and adjust values for your environment. Never commit your real `.env` file or credentials.

3. Start the services

The repository includes a Docker Compose configuration for PostgreSQL, the backend, worker, and frontend.

```
docker compose up --build
```

The Compose configuration maps the services to these local ports:

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:8000`
- PostgreSQL: `localhost:5432`

If the backend exposes interactive API documentation, check `http://localhost:8000/docs`.

Model configuration note: the Compose file references an Ollama-compatible endpoint, while the local Ollama service is behind the `local-llm` profile. To use that service, start the profile:

```
docker compose --profile local-llm up --build
```

You may also configure `LLM_BASE_URL`, `LLM_API_KEY`, and the model names in your environment for a compatible hosted or self-managed endpoint. Ensure the selected models are actually available from that endpoint. Starting the Ollama container alone does not guarantee the configured models have been downloaded or are ready.

4. Stop the services

```
docker compose down
```

To remove the local database volume as well, use `docker compose down -v`. **This deletes persisted local database data**, so use it only when that is intended.

Configuration

The `.env.example` file documents the main configuration variables, including:

Variable	Purpose
`ENVIRONMENT`	Runtime environment
`BACKEND_PORT`	Backend port setting
`FRONTEND_PORT`	Frontend port setting
`DATABASE_URL`	PostgreSQL connection string
`SECRET_KEY`	Application signing/secret key
`LLM_BASE_URL`	OpenAI-compatible model API endpoint
`LLM_API_KEY`	Credential for the model endpoint, if required
`LLM_AGENT_MODEL`	Configured agent model name
`LLM_VISION_MODEL`	Configured vision model name
`EMBED_MODEL`	Configured embedding model name
`WORKER_POLL_INTERVAL_SECONDS`	Worker polling interval
`MAX_TASK_ATTEMPTS`	Maximum configured task attempts

Use strong, unique secrets outside local development. Do not use sample credentials in a public deployment. Keep production credentials in a secrets manager or deployment environment configuration.

Development and testing

Before submitting a change:

- Read the relevant backend, frontend, or worker code before modifying it.
- Install dependencies using the instructions and dependency files for the area you are changing.
- Run the relevant tests and linters available in the repository.
- Test the full workflow affected by your change, including failure cases where practical.
- Update documentation when configuration, commands, API behaviour, or user-facing functionality changes.

The precise test commands depend on the current scripts and dependency configuration. Check the repository's `pyproject.toml`, requirements files, frontend `package.json`, and test configuration before running commands.

Security and privacy

CivicLoop may handle complaint references, photographs, locations, and other potentially sensitive information. Contributors should:

- Never commit API keys, passwords, private keys, tokens, or real citizen records.
- Use synthetic or appropriately anonymised data for examples and tests.
- Keep evidence files private and avoid exposing uploads through public routes.
- Validate inputs and enforce access control on the server, not only in the UI.
- Treat uploaded files and extracted text as untrusted data.
- Keep AI-generated suggestions distinct from verified facts and backend-confirmed state changes.
- Require explicit approval and an authorised integration before consequential external actions.
- Report suspected vulnerabilities privately to the repository maintainer rather than publishing exploit details in an issue.

Contributing

Contributions are welcome through the repository's normal review process.

- Open an issue or discussion first for substantial changes, if the repository's maintainers use them.
- Create a focused branch from the current default branch.
- Make a small, clearly scoped change.
- Add or update tests and documentation where relevant.
- Check that secrets and personal data are not included in the diff.
- Open a pull request explaining the problem, the change, how it was tested, and any known limitations.

Suggested branch names:

```
feat/short-feature-name
fix/short-bug-description
docs/short-documentation-change
```

Please preserve the project's emphasis on source traceability, human approval, permission checks, reliable follow-up, and privacy.

Acknowledgements

CivicLoop is a collaborative project maintained in the Rishank-07/Civic-Loop repository. Credit individual contributors for their actual work through Git history and merged pull requests; this README does not assign individual authorship or claim contributions that have not been verified.

License

No license is asserted by this README. Check the repository for a `LICENSE` file or confirm the intended licence with the project owner before redistributing or reusing the code.